DO NOW — RECALL

Lesson 116 · Linkers, Loaders and Program Libraries

Name: _____ Class: _____ Date: _____

1. Which compilation stage converts source characters into tokens?

2. In a BST, where is a value less than the current node placed?

3. Why can an array provide direct indexed access?

4. A full adder receives A=1, B=1 and carry-in=0. What are sum and carry-out?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Lexical analysis.

2. Model answer

In its left subtree (under the chosen no-duplicate convention).

3. Model answer

Its element locations can be calculated from base address and index.

4. Model answer

Sum 0 and carry-out 1.

By the end, you can…

• Source files become object modules with symbols/references.

• Linker resolves symbols and combines modules/libraries; static/dynamic linkage differs.

• Loader maps executable and needed libraries into memory, then begins execution.

Vocabulary — know these terms

linker — Resolves symbols and combines object modules.

loader — Places an executable in memory ready to run.

library — Reusable compiled routines/functions.

dynamic linking — Resolution/use of shared code at load time or runtime.

NEW LEARNING

Read, annotate and explain

Compile modules

Source files become object modules with symbols/references.

Link dependencies

Linker resolves symbols and combines modules/libraries; static/dynamic linkage differs.

Load and run

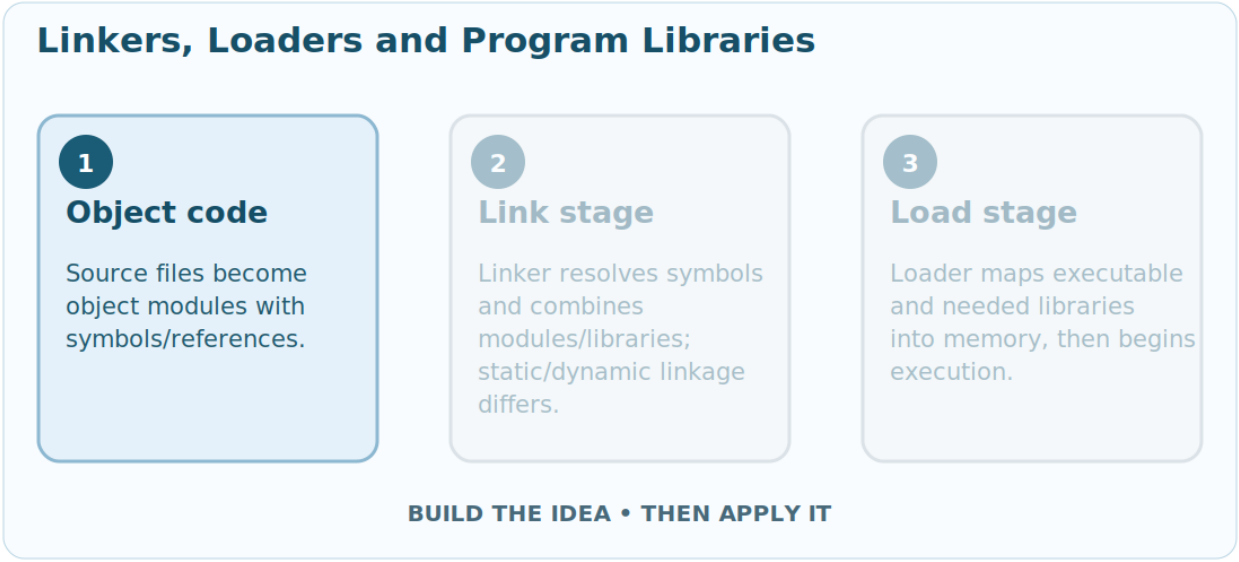
Loader maps executable and needed libraries into memory, then begins execution.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

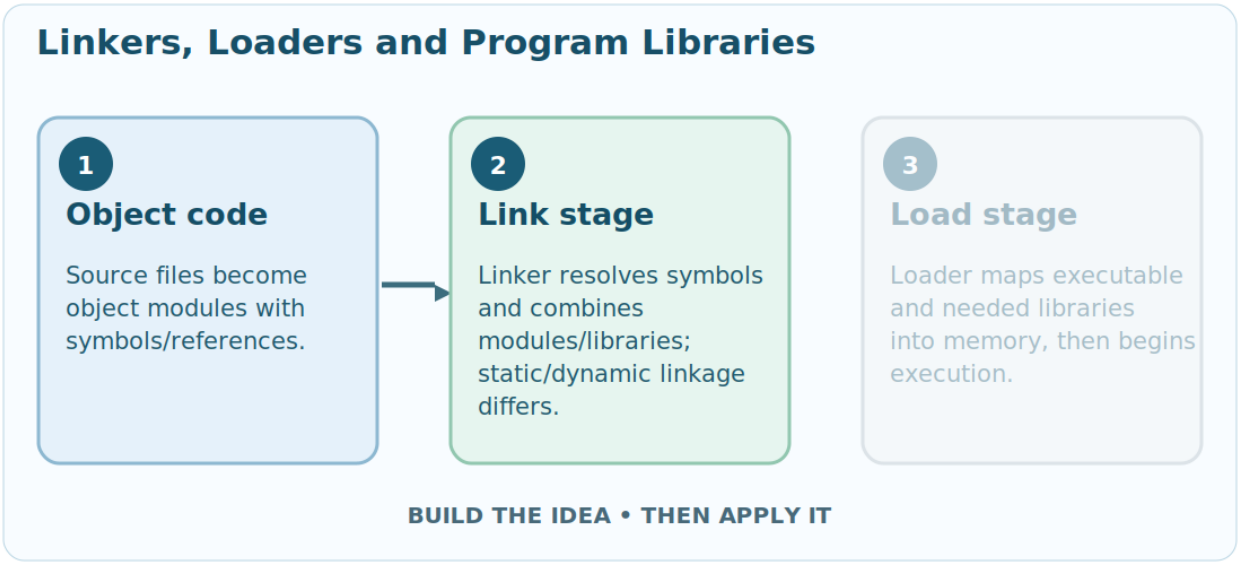
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Linkers, Loaders and Program Libraries: Object code then Link stage then Load stage

New Learning

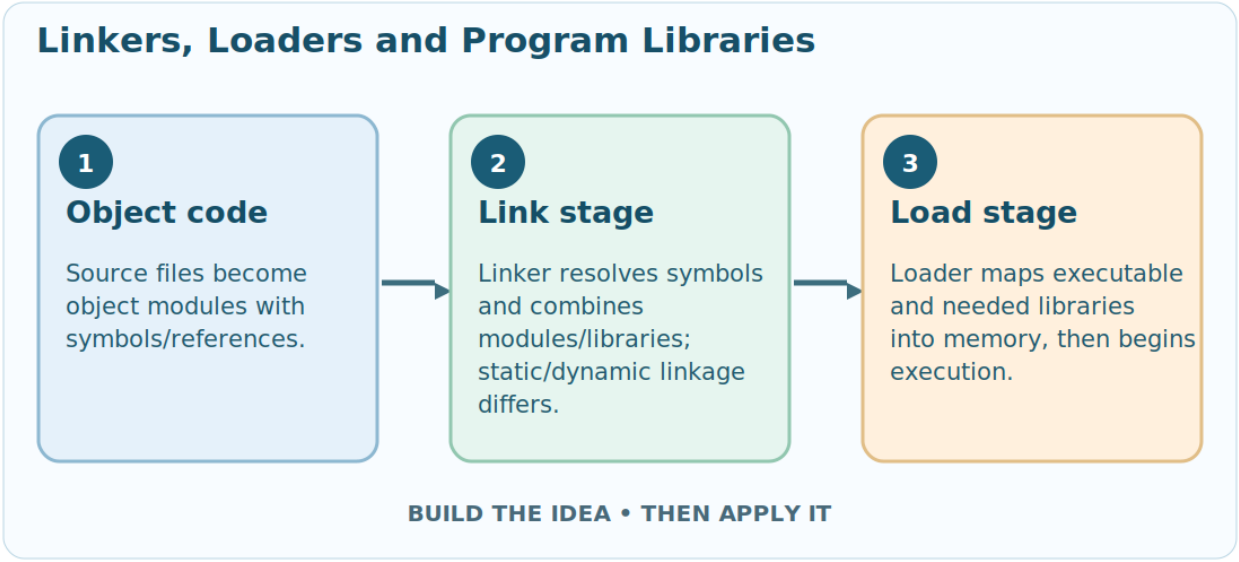


Linkers, Loaders and Program Libraries: Object code then Link stage then Load stage

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Linkers, Loaders and Program Libraries: Object code then Link stage then Load stage

I DO — WORKED EXAMPLE

Follow the worked method

main.c calls a function in maths.c. Explain the steps from separate source files to a running program.

Model answer

Compile source modules into object files; linker resolves the call and produces linked executable (or dynamic references); loader places executable and required libraries in memory and starts execution.

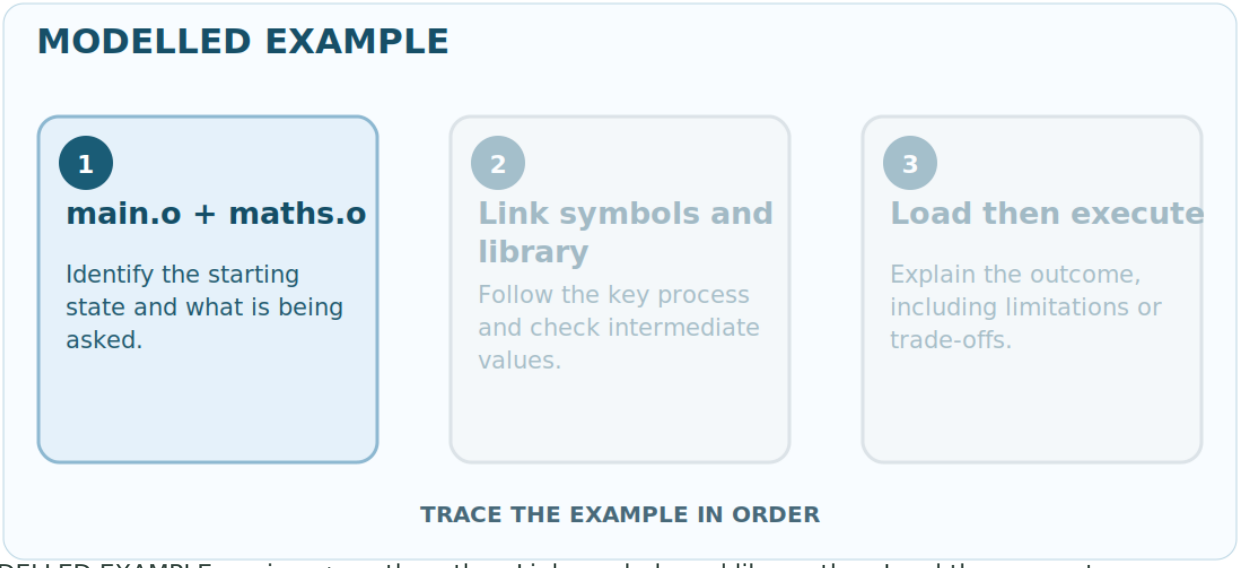
Check: link each claim to the scenario and use precise OCR terminology.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

main.c calls a function in maths.c. Explain the steps from separate source files to a running program.

Compile source modules into object files; linker resolves the call and produces linked executable (or dynamic references); loader places executable and required libraries in memory and starts executio



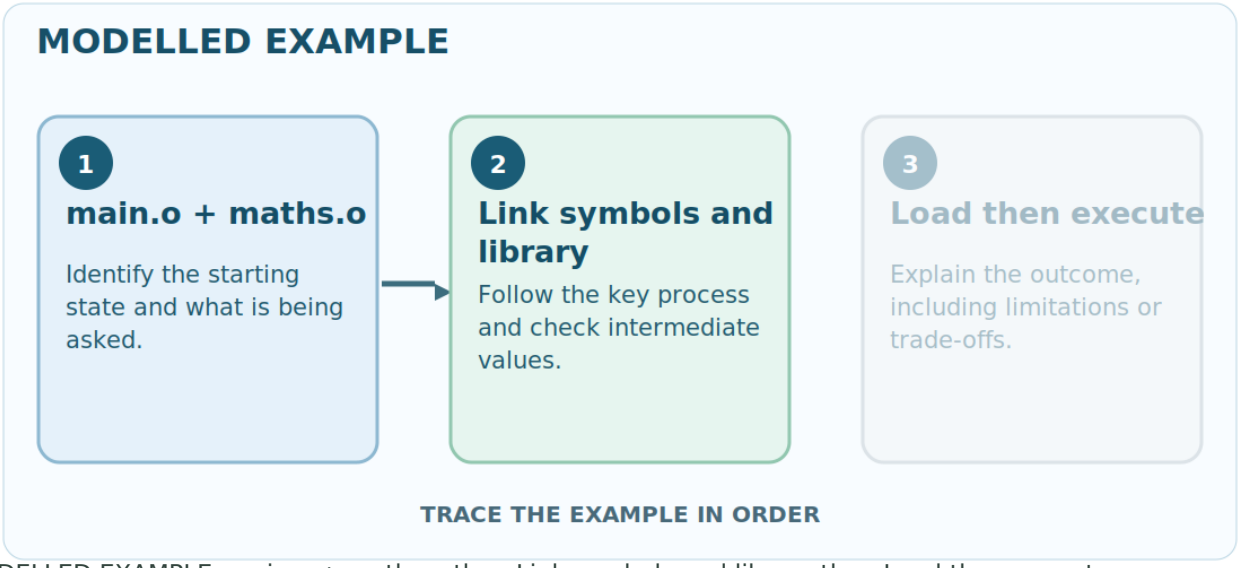
MODELLED EXAMPLE: main.o + maths.o then Link symbols and library then Load then execute

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

main.c calls a function in maths.c. Explain the steps from separate source files to a running program.

Compile source modules into object files; linker resolves the call and produces linked executable (or dynamic references); loader places executable and required libraries in memory and starts executio



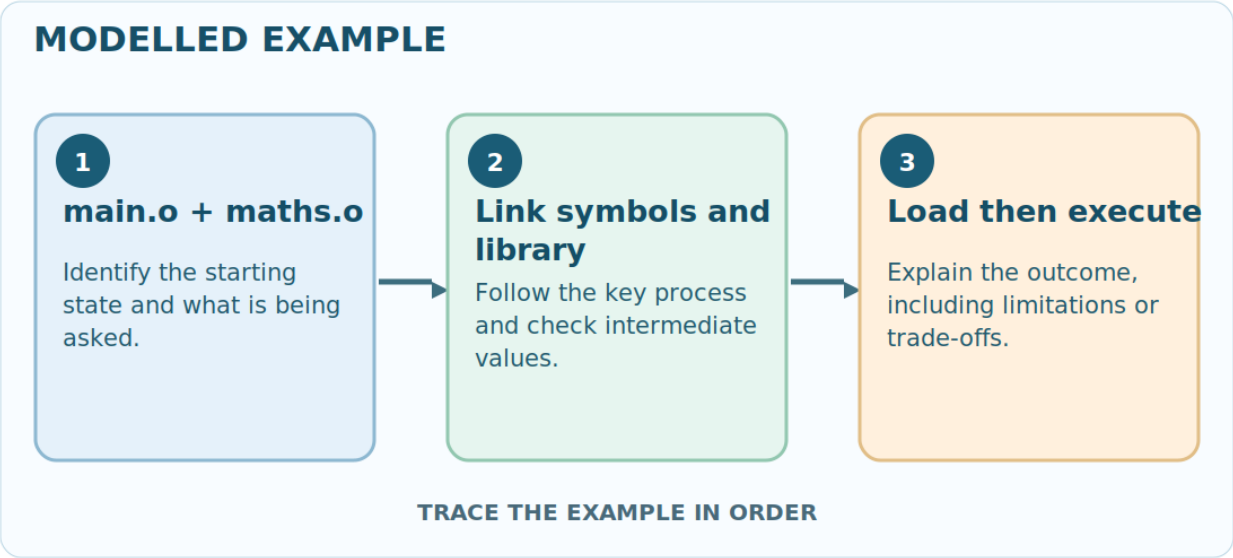
MODELLED EXAMPLE: main.o + maths.o then Link symbols and library then Load then execute

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

main.c calls a function in maths.c. Explain the steps from separate source files to a running program.

Compile source modules into object files; linker resolves the call and produces linked executable (or dynamic references); loader places executable and required libraries in memory and starts executio



MODELLED EXAMPLE: main.o + maths.o then Link symbols and library then Load then execute

WE DO — GUIDED PRACTICE

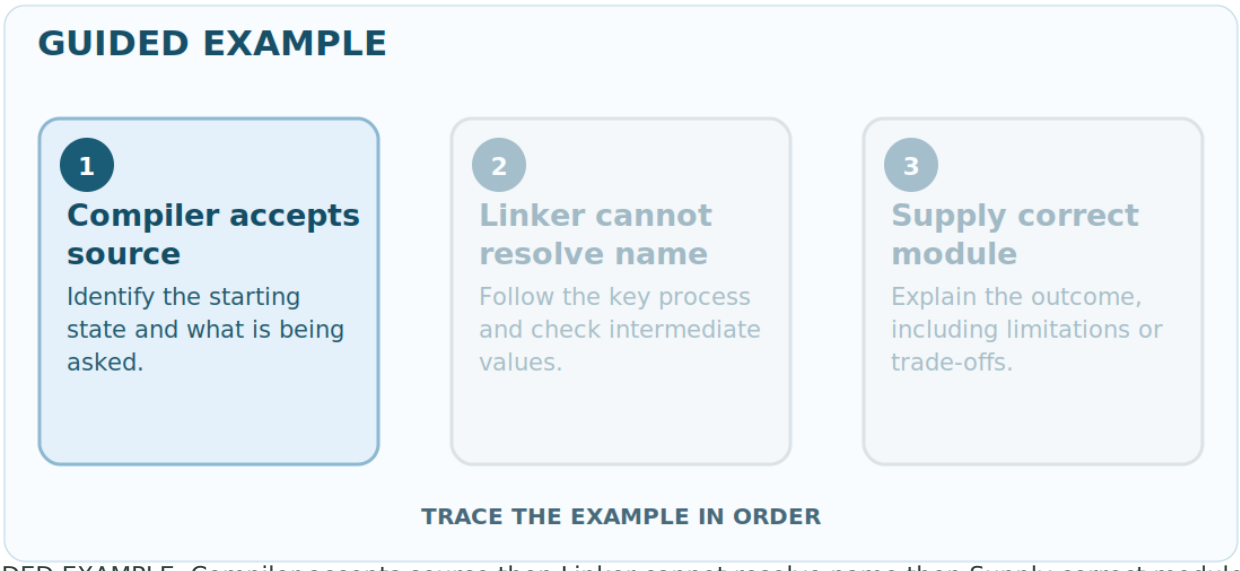
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A program compiles but the linker reports 'undefined reference: calculateTax'. Diagnose the fault.

Identify the inputs, relevant facts and exact output or judgement required.



GUIDED EXAMPLE: Compiler accepts source then Linker cannot resolve name then Supply correct module

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: What must we decide? Identify the inputs, relevant facts and exact output or judgement required.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Explain the roles of linker and loader with examples. [6]

Question 2

Distinguish static from dynamic linking, giving one trade-off of each. [6]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

A deployed app fails because a required shared library is unavailable. Explain why compile success did not prevent this. [8]

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A successful compilation guarantees that every dynamically linked library will be available at runtime.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

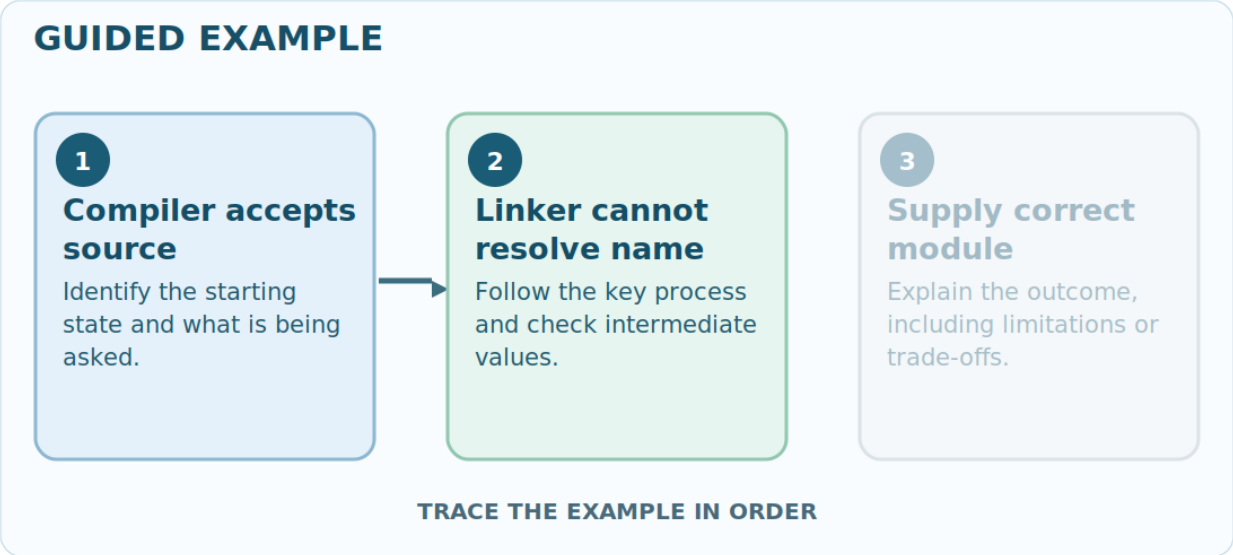
Next step: revisit the model or a relevant revision-guide section if needed.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A program compiles but the linker reports 'undefined reference: calculateTax'. Diagnose the fault.

Identify the inputs, relevant facts and exact output or judgement required.



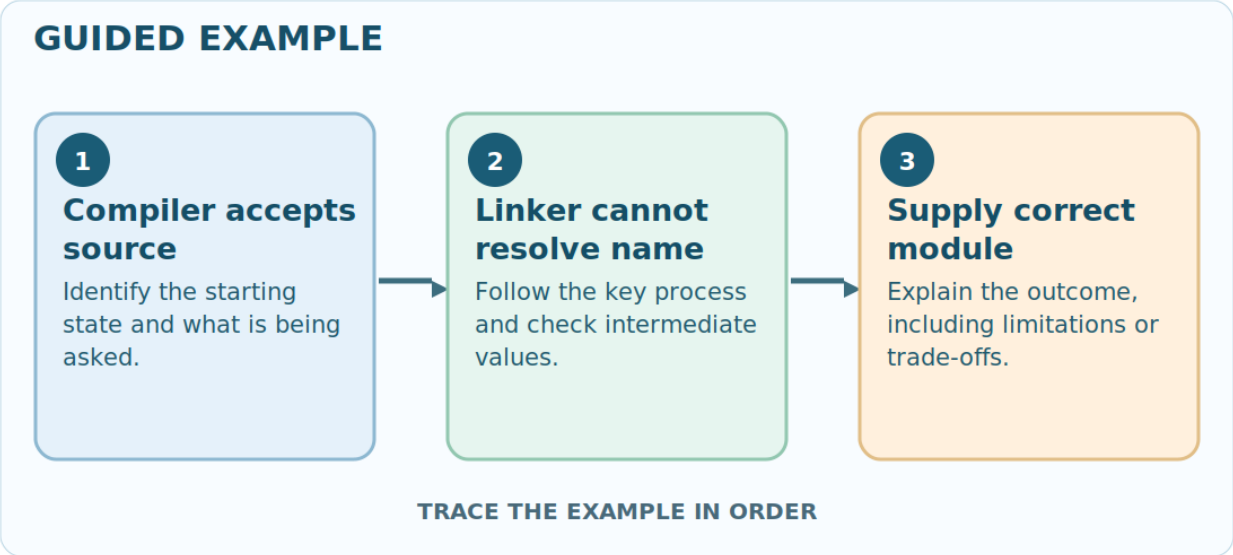
GUIDED EXAMPLE: Compiler accepts source then Linker cannot resolve name then Supply correct module

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A program compiles but the linker reports 'undefined reference: calculateTax'. Diagnose the fault.

Identify the inputs, relevant facts and exact output or judgement required.



GUIDED EXAMPLE: Compiler accepts source then Linker cannot resolve name then Supply correct module

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Work through the method

Use the middle stage of the visual to follow the process and state each intermediate result.

Shared worked answer

Lexical and syntax stages succeeded. The linker cannot find the referenced symbol's definition: implementation may be missing, not linked, misspelled or incompatible. Add the correct object/library or fix the symbol.

IMPROVE: Explain one correction to your own reasoning.